Clean Code + Time & Space Complexity

What is Clean Code?

- Code that is **easy to read, understand, modify, and debug**.
- Bad code works today.
- Clean code works **today + tomorrow + for others**.

1. Meaningful Variable Names

- **BAD**

```
x = 10
y = 20
z = x + y
```

- **CLEAN**

```
price = 10
tax = 20
total_cost = price + tax
```

- **Rule:**
  - Names should explain *why* the variable exists.

2. Follow One Job per Function (SRP)

- **BAD**

```python
def process_data(data):
    print(data)
    total = sum(data)
    return total
```

- **CLEAN**

```python
def display_data(data):
    print(data)

def calculate_total(data):
    return sum(data)
```

- **Rule:**
  - One function = one responsibility

3. Avoid Hard-Coding Values

- **BAD**

```
salary = hours * 500
```

- **CLEAN**

```
HOURLY_RATE = 500
salary = hours * HOURLY_RATE
```

- **Rule:**
  - Use constants for fixed values.

4. Keep Code Readable (Spacing & Style)

- **BAD**

```
if(x>5):print("Hi")
```

- **CLEAN**

```
if x > 5:
    print("Hi")
```

- **Rule:**
  - Code is read more than it’s written.

5. Comment *Why*, Not *What*

- **BAD**

```
# add two numbers
total = a + b
```

- **CLEAN**

```
# Apply discount before tax calculation
total = discounted_price + tax
```

Practice

```
def f(a,b):
    return a*b*12
```